Functional Documentation — Anonify Platform

Purpose

Anonify is a secure enterprise data migration and masking platform designed to move data from protected source systems into non-production environments while applying masking during the migration process.

The repository structure shows that the platform is implemented as a Spring Boot application with controllers, services, batch components, models, repositories, and deployment artifacts that support migration orchestration and governance.

Functional Objective

The functional objective of the platform is to let business or technical teams provision realistic non-production data without exposing raw sensitive information from production or secure databases.

The application is designed to support controlled configuration, review, approval, scheduling, and execution of migration jobs across relational and MongoDB-oriented workloads.

Core Functional Theme

The central theme of the project is data protection through three transformation methods applied during migration.

Standard Mocking

Standard mocking replaces original values with realistic test data using faker-style generation. This approach is appropriate for values such as names, emails, phone numbers, or descriptive business fields where realism is useful but exact value continuity is not required.

Partial Masking

Partial masking hides most of a value while preserving a small visible suffix, typically the last four characters or digits. This allows test users to identify or verify a record without exposing the complete sensitive value.

Format Preserving Hashing

Format Preserving Hashing is intended for primary keys, foreign keys, and unique values that must remain consistently transformed across tables. The repository contains dedicated encryption and masking services, and the project design emphasizes deterministic transformation so related values such as `dept_id` can remain join-compatible across employee, salary, and department tables after masking.

This function is critical because referential integrity must be preserved when the same business key appears in multiple tables. A consistent transformed value ensures application logic, queries, and validation processes can still use table relationships safely in the target environment.

Functional Scope

The current repository indicates that the platform covers authentication, connection management, masking configuration, migration orchestration, batch execution, MongoDB handling, approvals, and file-system or storage-related operations.

This scope supports a full governed workflow from migration setup through execution and audit-oriented tracking.

Functional Modules

Module	Functional Description
Authentication and user access	Supports user login and role-driven access handling.
Department and connection setup	Supports creation and management of database and storage connection definitions.
Table and collection discovery	Supports discovery of source and destination structures before migration.

Masking configuration	Supports definition and application of masking rules.
Migration job execution	Supports creation and execution of migration jobs through service and batch components.
Approval workflow	Supports review and approval or rejection of configured migration tasks.
Operational tracking	Supports job, checkpoint, and audit-related state through modeled entities.
MongoDB migration	Supports Mongo discovery, read, process, write, and collection cleanup operations.

User Roles

The project's functional workflow is built around three role types, supported by the repository's auth, approval, user, and role-related components.

Scheduler

The Scheduler is a department-based operational user who configures migration jobs. The Scheduler selects the source and destination database, chooses the tables or collections to migrate, defines which fields will use standard mocking, partial masking, or format preserving hashing, submits the configuration for approval, and schedules or runs the job once it has been approved.

The same business workflow is intended to apply not only to relational tables but also to MongoDB and JSON-style migration scenarios. This makes the Scheduler the main role responsible for preparing department-level data refreshes.

Approver

The Approver is also department-based and has a focused governance role. The Approver reviews the migration configuration and either approves or rejects it, but does not perform broader platform setup or execution activity.

This role enforces separation of duties between job preparation and job authorization. The approval-related controller and service components in the repository support this governed operating model.

Administrator

The Administrator has the broadest access and can perform both Scheduler and Approver responsibilities. In addition, the Administrator can create and assign database configurations, configure storage, create PostgreSQL and MongoDB connectivity, access dashboards, run jobs, and manage platform-wide operational settings.

This role represents central platform ownership. The repository's connection, file-system, approval, auth, and migration components align with this administrative capability model.

Department-Based Operating Model

The functional model is department-centric, where each department can have multiple databases and multiple connection definitions. Systems are mapped with source and destination pairs, where the source is typically a production or secure data store and the destination is a non-production environment used for testing, QA, development, or staging.

This model is important because it lets governance, access, and approval be applied within departmental boundaries while still using a common enterprise platform. It also supports scaling the platform across multiple business units without losing centralized administrative control.

End-to-End Functional Workflow

1. An Administrator creates and assigns database or storage configurations to departments using the platform's connection and configuration capabilities.
2. A Scheduler selects the department context and chooses the source system and destination environment for a migration task.
3. The Scheduler selects the list of tables, collections, or supported data assets to migrate.
4. The Scheduler chooses field-level treatment for each selected dataset, including standard mocking, partial masking, or format preserving hashing.
5. The Scheduler submits the migration configuration to the department Approver.
6. The Approver reviews the request and either approves or rejects the migration configuration.
7. After approval, the Scheduler or Administrator schedules or runs the migration job.

8. The application executes the migration using service-layer orchestration and Spring Batch execution components such as readers, processors, writers, and tasklets.
9. Job, checkpoint, and audit information is retained for operational tracking and governance review.

Relational and MongoDB Functionality

The platform is not limited to standard relational-table migration. The repository includes explicit MongoDB dependencies and Mongo-specific discovery, migration, reader, processor, writer, progress, and cleanup components, showing that MongoDB handling is part of the supported functional scope.

The project direction also extends the same masking and approval concept to JSON-oriented migration scenarios. This means the business workflow remains consistent across different data structures even when the storage model changes.

Functional Requirements

Job Configuration

- The platform shall allow a Scheduler to create a migration job for a department.
- The platform shall allow selection of a source and destination connection.
- The platform shall allow selection of tables, collections, or supported data assets to migrate.
- The platform shall allow field-level selection of masking method.
- The platform shall allow the job configuration to be submitted for approval before execution.

Masking and Transformation

- The platform shall support standard mocking for general business fields.
- The platform shall support partial masking for sensitive values where limited visibility is required.
- The platform shall support deterministic format preserving hashing for primary keys, foreign keys, and unique values that must remain consistent across related tables.
- The platform shall preserve relational usability where the same constrained key appears in multiple datasets.

Approval and Execution

- The platform shall allow an Approver to review a configured migration task.
- The platform shall allow an Approver to approve or reject the migration request.
- The platform shall allow only approved migration tasks to be scheduled or executed.
- The platform shall support operational execution using Spring Batch-oriented components.

Administration

- The platform shall allow an Administrator to create and assign database configurations to departments.
- The platform shall allow an Administrator to configure storage options including cloud object storage and local storage, consistent with the project direction and file-system/COS related components in the repository.
- The platform shall allow an Administrator to configure PostgreSQL and MongoDB connectivity for migration jobs.
- The platform shall allow an Administrator to access dashboards, schedule jobs, run jobs, and manage platform-wide settings.

Technology Context

The implementation uses Spring Boot 3.5.4 as the backend framework, Java 17 as the runtime target in the current build, Spring Batch for execution, and Spring Data modules for relational and MongoDB integration.

The broader project direction includes dashboarding with HTML, CSS, JavaScript, and Tailwind-based graph presentation. A management note is that the README also mentions a future-oriented stack vision using newer framework versions, but the checked-in Maven configuration currently reflects Spring Boot 3.5.4 and Java 17.

Data Integrity Function

One of the most important functional outcomes of the platform is maintaining data integrity after masking. The project design explicitly emphasizes deterministic transformation for related keys so tables continue to join correctly after migration, which is essential for test accuracy and application behavior validation.

Without this function, masked data would lose much of its business usefulness because relationships between master and transactional data could break. The format preserving hashing approach is therefore not only a security feature but also a core functional requirement for enterprise-quality testing.

Audit and Governance Function

The repository includes models such as audit logs, job checkpoints, job definitions, users, roles, and migration-related entities, showing that governance is built into the functional design.

This enables the platform to support controlled execution, reviewability, and restart-oriented operations rather than simple one-time data copy activity.

For higher-level governance, this means the platform can support evidence-based review of who configured a job, whether it was approved, and how execution progressed. That makes the product suitable for controlled enterprise data operations instead of informal refresh scripts.

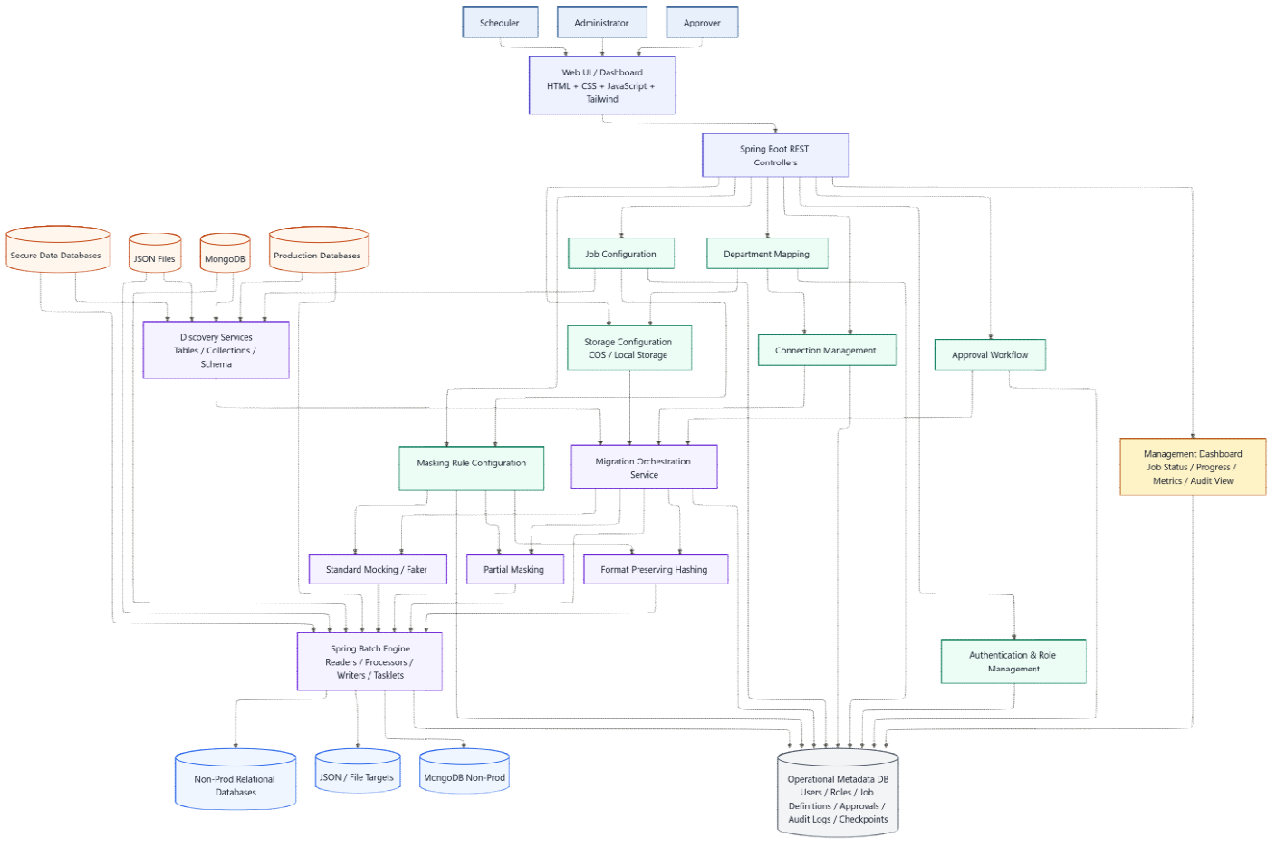
Summary of Functional Value

Functionally, Anonify combines data selection, field-level transformation, approval workflow, role-based governance, and controlled execution into one platform. The current repository provides strong implementation evidence for that model through its service, controller, batch, and domain structure.

Its key business value is that it allows organizations to refresh lower environments with structurally useful data while protecting sensitive information and maintaining inter-table relationships needed for meaningful testing.

Anonify Architecture Diagram

The Anonify platform is a department-governed data masking and migration system with role-based workflow, configurable data sources and destinations, masking services, and a Spring Batch execution core backed by operational metadata.



Color Legend

- Light blue represents business users and access actors.
- Indigo represents the presentation and API layer.
- Green represents governance and configuration controls.
- Orange represents source systems.
- Purple represents core masking and migration processing services.
- Blue represents destination environments.
- Gray represents metadata and audit persistence.
- Gold represents executive and operational dashboarding.

Diagram Notes

This version uses a more professional enterprise color palette to visually separate user roles, application layers, source systems, processing components, targets, and metadata management. The structure still reflects the same Anonify functional model of Scheduler, Approver, and Administrator access flowing through Spring Boot controllers into configuration, approval, masking, orchestration, and Spring Batch execution.

The diagram also highlights the relationship between discovery, masking-rule application, deterministic hashing, and target delivery so management can clearly see how protected data moves from secure sources into non-production destinations while operational tracking is retained in metadata tables.